

Clase_06_Regresión_JorgeLuisGuzman

May 17, 2026

1 Regresión lineal

En este notebook trabajaremos con datos simulados para estudiar:

- cómo se ajusta una regresión lineal
- cómo se evalúa un modelo de regresión
- cómo cambia el ajuste frente a outliers
- cómo funciona Gradient Descent
- cómo Ridge, Lasso y Elastic Net modifican los coeficientes.

Basado en Machine learning for physics and Astronomy, Viviana Acquaviva (2023)

1.1 Instrucciones

Este notebook combina celdas demostrativas con preguntas de análisis.

No basta con ejecutar el código: deben interpretar los resultados, comparar métricas y justificar sus respuestas usando gráficos, coeficientes y residuos.

```
[1]: import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import sklearn
from sklearn import metrics
from sklearn.model_selection import train_test_split, cross_validate, \
    cross_val_predict
from sklearn.model_selection import KFold
from sklearn import linear_model # Nuevo modelo desbloqueado

font = {'size' : 16}
matplotlib.rc('font', **font)
matplotlib.rc('xtick', labels=14)
matplotlib.rc('ytick', labels=14)
#matplotlib.rcParams.update({'figure.autolayout': False})
matplotlib.rcParams['figure.dpi'] = 100
```

```
[2]: from sklearn.metrics._scorer import _SCORERS
```

Generamos un dataset de juguete Primero trabajaremos con datos simulados. La ventaja es que conocemos la relación verdadera:

$$y_{\text{true}} = 3x + 3$$

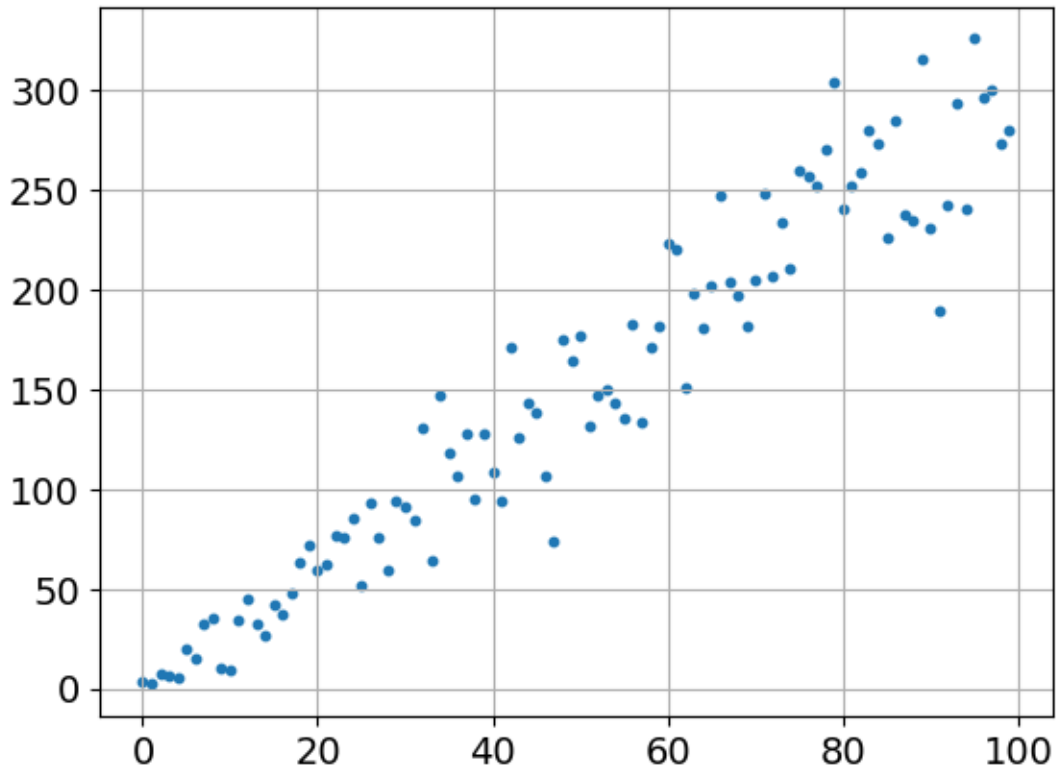
Luego agregamos ruido para simular mediciones imperfectas.

```
[3]: np.random.seed(16) #fijar semilla para la reproducibilidad

x = np.arange(100)

yp = 3*x + 3 + 2*(np.random.poisson(3*x+3,100)-(3*x+3))
#genera datos con dispersión, siguiendo una distribución Poissoniana
# con valor esperado = y (modelo lineal), centrado alrededor de cero

[4]: plt.scatter(x, yp, s = 10);
plt.grid()
```



1.1.1 Pregunta

De acuerdo al gráfico anterior: - ¿Los datos siguen una tendencia aproximadamente lineal? - ¿La dispersión alrededor de la recta parece constante para todos los valores de (x)? - ¿Qué podría implicar esto al mirar residuos más adelante?

1.1.2 Respuesta

¿Los datos siguen una tendencia aproximadamente lineal?

Sí. El gráfico muestra que los puntos siguen, en promedio, una tendencia creciente coherente con la relación verdadera $y_{\text{true}} = 3x + 3$. La regresión lineal es un modelo apropiado para estos datos.

¿La dispersión alrededor de la recta parece constante para todos los valores de x ?

No. La dispersión crece notablemente con x : para valores pequeños los puntos están muy concentrados alrededor de la línea, mientras que para valores grandes se alejan mucho más. Esto se debe a que el ruido fue generado con una distribución de Poisson, cuya varianza es igual a su media ($\lambda = 3x + 3$), por lo que la varianza también crece con x .

¿Qué podría implicar esto al mirar los residuos más adelante?

Se espera observar que la dispersión de los residuos no será constante sino que aumentará con x . Esto se contradice con el supuesto de varianza constante de la regresión lineal ordinaria, aunque no impide que el modelo ajuste bien la tendencia promedio.

```
[5]: model = linear_model.LinearRegression()
```

Ajuste con regresión lineal Ajustaremos un modelo de la forma:

$$\hat{y} = \beta_0 + \beta_1 x$$

En `sklearn`, cuando tenemos una sola feature, necesitamos escribir x como una matriz de forma $(n_{\text{samples}}, 1)$. Por eso usamos:

```
“python x.reshape(-1, 1)
```

```
[6]: model.fit(x.reshape(-1, 1), yp)
```

```
[6]: LinearRegression()
```

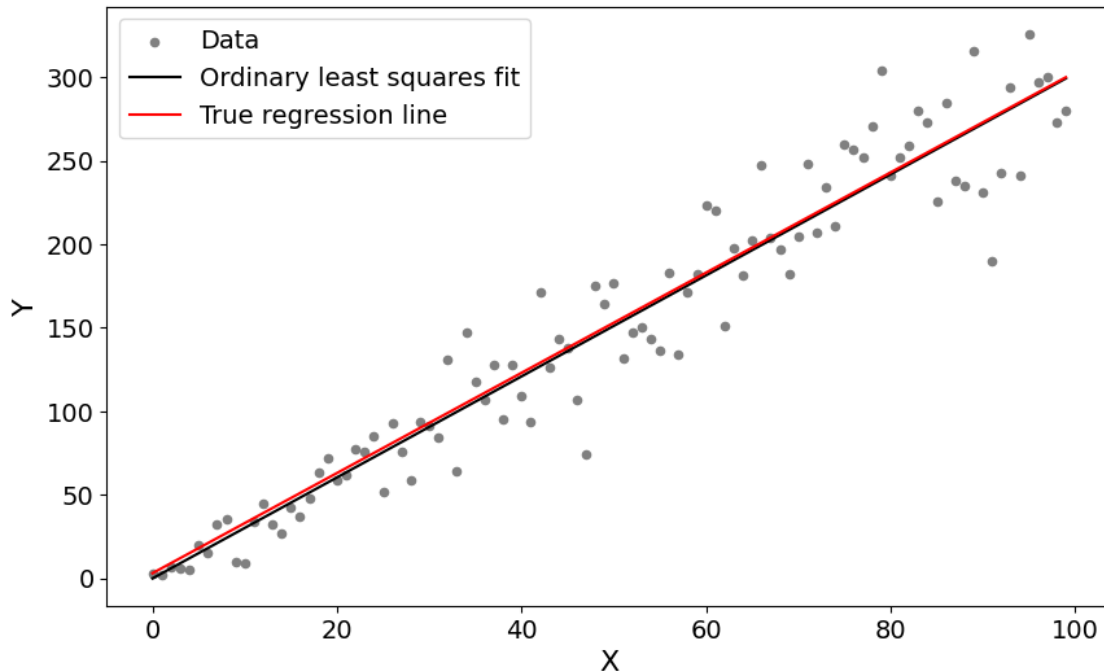
```
[7]: slope = model.coef_[0]
      intercept = model.intercept_

      print(f"Intercepto beta_0 = {intercept:.3f}")
      print(f"Pendiente beta_1 = {slope:.3f}")
```

```
Intercepto beta_0 = -0.126
```

```
Pendiente beta_1 = 3.025
```

```
[8]: plt.figure(figsize = (10,6))
      plt.scatter(x,yp, s = 20, c = 'gray', label = 'Data')
      plt.plot(x, slope*x + intercept, c='k', label = 'Ordinary least squares fit')
      plt.plot(x, 3*x + 3, c = 'r', label = 'True regression line')
      plt.legend(fontsize = 14)
      plt.xlabel('X')
      plt.ylabel('Y')
      plt.show()
```



1.1.3 Pregunta

Compara los coeficientes ajustados con los valores verdaderos $\beta_0 = 3$ y $\beta_1 = 3$ - ¿Son exactamente iguales? ¿Por qué? - ¿Qué representa físicamente/matemáticamente la pendiente en este modelo?

1.1.4 Respuesta

¿Son exactamente iguales los coeficientes ajustados a los valores verdaderos?

No exactamente. Se obtuvo $\hat{\beta}_0 = -0.126$ e $\hat{\beta}_1 = 3.025$, mientras que los valores verdaderos son $\beta_0 = 3$ y $\beta_1 = 3$. La pendiente es muy cercana (error $< 1\%$), pero el intercepto difiere más. Esto ocurre porque el ajuste se realiza sobre datos que incluyen **ruido aleatorio**: los mínimos cuadrados minimizan el MSE sobre la muestra particular generada, no sobre los valores exactos del modelo verdadero. Con otra semilla aleatoria o más datos, los coeficientes convergerían más hacia los valores verdaderos.

¿Qué representa físicamente/matemáticamente la pendiente?

La pendiente $\hat{\beta}_1 \approx 3.025$ indica que, por cada incremento de una unidad en x , el valor esperado de y aumenta en aproximadamente 3.025 unidades. Es la **tasa de cambio media** de la variable respuesta respecto al predictor. Matemáticamente, es la derivada de $\hat{y}$ respecto a x en el modelo lineal ajustado.

Solución analítica de mínimos cuadrados Para regresión lineal simple, la pendiente que minimiza el MSE puede escribirse como:

$$\beta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

y luego:

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

Implemente en el código para calcular a partir de la solución analítica los coeficientes y compare

```
[9]: beta0= yp.mean() - slope*x.mean()
beta1= np.sum((x - x.mean())*(yp - yp.mean()))/np.sum((x - x.mean())**2)
print(f"Beta_0 analítico = {beta0:.3f}")
print(f"Beta_1 analítico = {beta1:.3f}")
```

Beta_0 analítico = -0.126

Beta_1 analítico = 3.025

Evaluación del modelo: validación cruzada y métricas Ahora aplicamos una idea que ya usamos en clasificación: validar el modelo en distintos subconjuntos de datos.

Por defecto, `LinearRegression.score()` devuelve (R^2), no accuracy.

Podemos ver todos los scorers implementados en sklearn

```
[10]: from sklearn.metrics import get_scorer_names
print(get_scorer_names())

['accuracy', 'adjusted_mutual_info_score', 'adjusted_rand_score',
'average_precision', 'balanced_accuracy', 'completeness_score',
'd2_absolute_error_score', 'd2_brier_score', 'd2_log_loss_score',
'explained_variance', 'f1', 'f1_macro', 'f1_micro', 'f1_samples', 'f1_weighted',
'fowlkes_mallows_score', 'homogeneity_score', 'jaccard', 'jaccard_macro',
'jaccard_micro', 'jaccard_samples', 'jaccard_weighted', 'matthews_corrcoef',
'mutual_info_score', 'neg_brier_score', 'neg_log_loss', 'neg_max_error',
'neg_mean_absolute_error', 'neg_mean_absolute_percentage_error',
'neg_mean_gamma_deviance', 'neg_mean_poisson_deviance',
'neg_mean_squared_error', 'neg_mean_squared_log_error',
'neg_median_absolute_error', 'neg_negative_likelihood_ratio',
'neg_root_mean_squared_error', 'neg_root_mean_squared_log_error',
'normalized_mutual_info_score', 'positive_likelihood_ratio', 'precision',
'precision_macro', 'precision_micro', 'precision_samples', 'precision_weighted',
'r2', 'rand_score', 'recall', 'recall_macro', 'recall_micro', 'recall_samples',
'recall_weighted', 'roc_auc', 'roc_auc_ovo', 'roc_auc_ovo_weighted',
'roc_auc_ovr', 'roc_auc_ovr_weighted', 'top_k_accuracy', 'v_measure_score']
```

Implemente una validación cruzada con Kfold y encuentre los resultados para las métricas R^2 , 'neg_mean_absolute_error' (MAE) y 'neg_mean_squared_error' (MSE)

Para los estimadores de la performance del modelo del tipo “error” siempre queremos que sean pequeños (menor error, mejor modelo). Pero en sklearn, reciben un signo negativo, como `neg_mean_squared_error`, para mantener la consistencia de “alto puntaje=mejor” de los scorers

```
[11]: kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

```
[12]: scores_r2 = cross_validate(model, x.reshape(-1, 1), yp, cv=kf, scoring='r2',  
    ↪return_train_score=True)  
print(f"R2 test : {scores_r2['test_score'].mean():.3f} ±  
    ↪{scores_r2['test_score'].std():.3f}")  
print(f"R2 train: {scores_r2['train_score'].mean():.3f} ±  
    ↪{scores_r2['train_score'].std():.3f}")
```

R2 test : 0.903 ± 0.059

R2 train: 0.925 ± 0.007

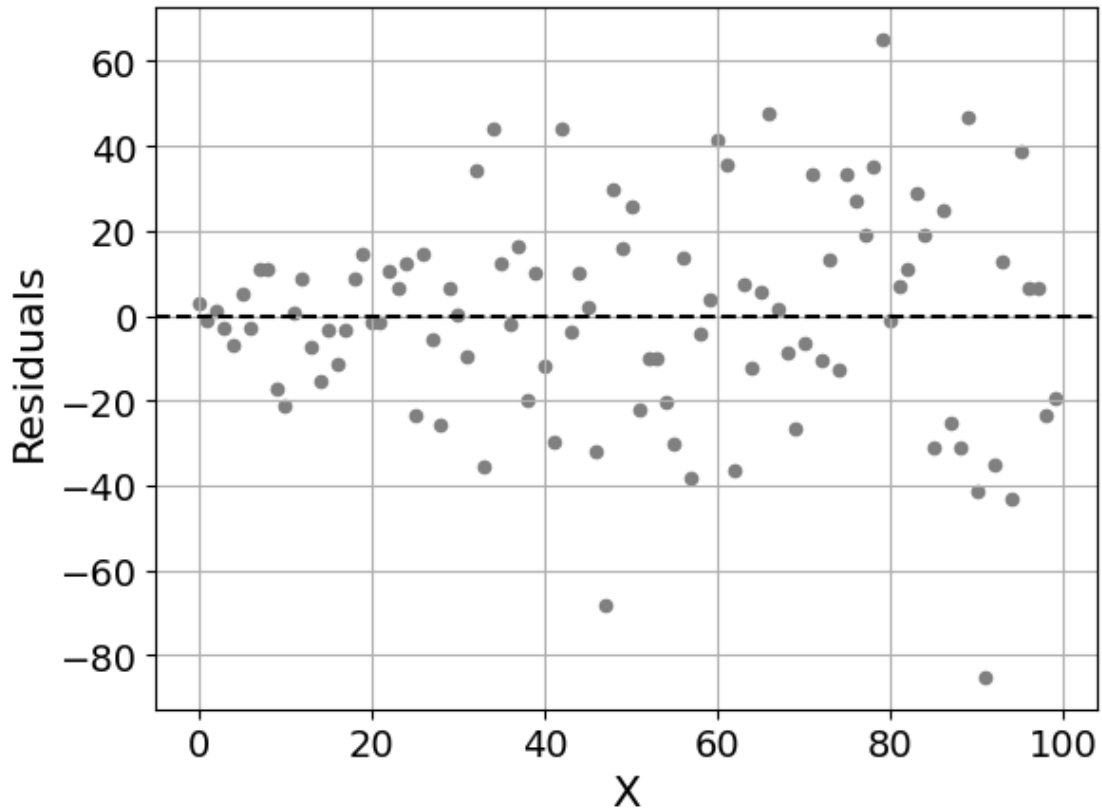
```
[13]: scores_mse = cross_validate(model, x.reshape(-1, 1), yp, cv=kf,  
    ↪scoring='neg_mean_squared_error', return_train_score=True)#implemente una  
    ↪validacion cruzada  
  
scores_mae = cross_validate(model, x.reshape(-1, 1), yp, cv=kf,  
    ↪scoring='neg_mean_absolute_error', return_train_score=True)#implemente una  
    ↪validacion cruzada  
  
print(f"MSE test: {-scores_mse['test_score'].mean():.3f}")  
print(f"MAE test: {-scores_mae['test_score'].mean():.3f}")
```

MSE test: 633.964

MAE test: 19.067

Haga un gráfico de los residuos $r_i = y_i - \hat{y}_i$

```
[14]: # grafico de los residuos r_i = y_i - y_hat_i  
y_pred = model.predict(x.reshape(-1, 1))  
residuals = yp - y_pred  
plt.scatter(x, residuals, s = 20, c = 'gray')  
plt.axhline(0, c = 'k', ls = '--')  
plt.xlabel('X')  
plt.ylabel('Residuals')  
plt.grid()  
plt.show()
```



Pregunta

- ¿Los residuos parecen distribuidos aleatoriamente alrededor de cero?
- ¿La dispersión de los residuos cambia con x ?
- ¿Qué nos dice esto sobre el modelo y/o sobre cómo fueron generados los datos?

Respuesta ¿Los residuos parecen distribuidos aleatoriamente alrededor de cero?

En términos de su valor medio sí: los residuos están centrados alrededor de cero a lo largo de todo el rango de x , lo que indica que el modelo captura correctamente la tendencia lineal sin sesgo sistemático.

¿La dispersión de los residuos cambia con x ?

Sí, claramente. La variabilidad de los residuos aumenta con x : para x pequeño los residuos son compactos, mientras que para x grande se dispersan mucho más.

¿Qué nos dice esto sobre el modelo y/o los datos?

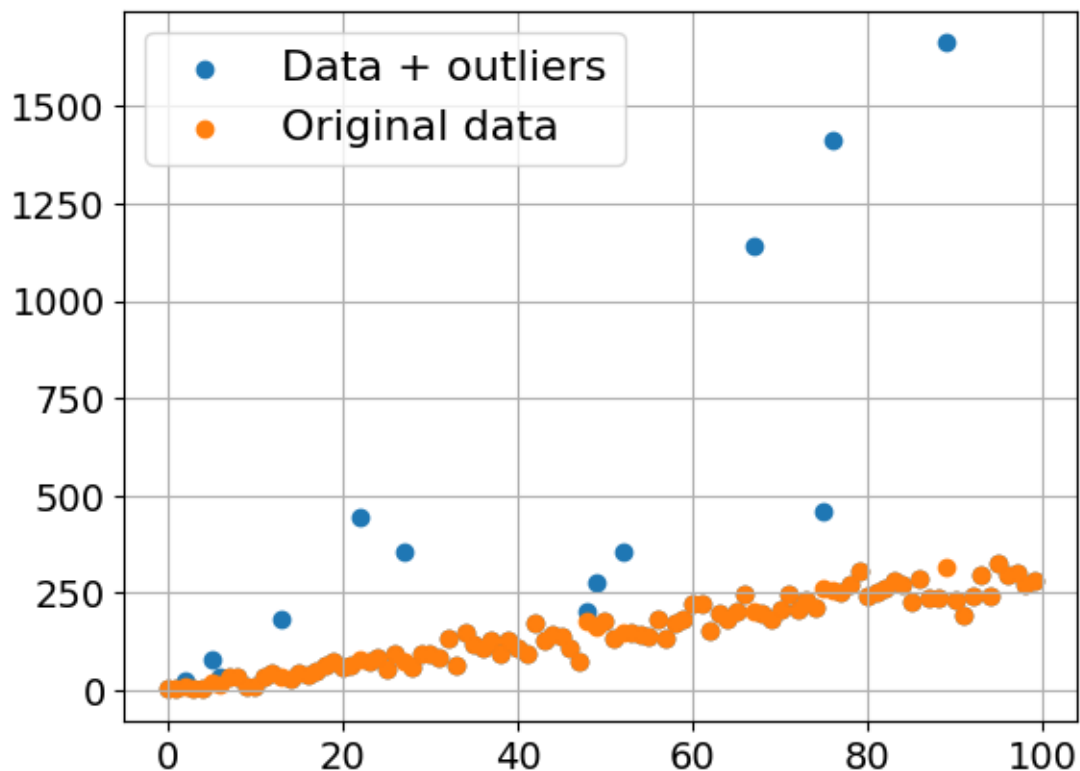
- El modelo lineal captura bien la **tendencia promedio** (media condicional), pero el supuesto de **varianza constante** no se cumple.
- Esto es consistente con la forma en que se generaron los datos: el ruido sigue una distribución de Poisson con $\lambda = 3x + 3$, cuya varianza es exactamente λ , creciendo linealmente con x .
- En la práctica, la heterocedasticidad no invalida el modelo, pero afecta la estimación de los errores

estándar de los coeficientes y los intervalos de confianza; podrían considerarse transformaciones (p.ej. $\sqrt{y}$) o modelos con pesos para mitigarla.

1.1.5 ¿Qué ocurre si agregamos outliers?

```
[15]: np.random.seed(12) # fijada la semilla
      out = np.random.choice(100,15) #seleccionamos 15 índices de outliers
      yp_wo = np.copy(yp)
      np.random.seed(12) #fijada
      yp_wo[out] = yp_wo[out] + 5*np.random.rand(15)*yp[out]
```

```
[16]: plt.scatter(x,yp_wo, label = 'Data + outliers')
      plt.scatter(x,yp, label = 'Original data')
      plt.legend();
      plt.grid()
```



Ajuste el modelo de regresión lineal para los datos con outliers Responda: - Compare visualmente el ajuste antes y después de agregar outliers. ¿La recta cambia mucho? ¿Hacia dónde se mueve? - Compara los coeficientes β_0 y β_1 del modelo con y sin outliers. ¿Cuál cambia más: el intercepto o la pendiente? - Compara las métricas MAE, MSE, RMSE y R^2 antes y después de agregar outliers. ¿Qué métrica se ve más afectada? - ¿Por qué el MSE/RMSE suele ser más sensible a outliers que el MAE? - ¿Todos los outliers afectan de la misma manera al modelo? Observa si los

puntos atípicos están lejos en y , lejos en x , o en ambos. - ¿Dirías que el modelo con outliers está aprendiendo la tendencia general de los datos o está siendo “tirado” por algunos puntos extremos? Justifica. - Si este fuera un dataset real, ¿eliminarías los outliers automáticamente? Explica qué revisarías antes de decidir.

Respuesta 1. Comparación visual del ajuste antes y después de outliers

La recta ajustada sobre datos con outliers se desplaza notoriamente respecto al ajuste original: la pendiente aumenta (de 3.025 a ~3.978) y el intercepto también cambia (de -0.126 a ~1.548). La línea se “tira” hacia los puntos atípicos, dejando de representar bien la tendencia del grueso de los datos.

2. ¿Cuál coeficiente cambia más: intercepto o pendiente?

Ambos cambian, pero el **intercepto** sufre una variación relativa mayor (de -0.126 a 1.548, un cambio de ~1.67 unidades absolutas), mientras que la pendiente aumenta de 3.025 a 3.978 (~+0.95). En términos relativos el intercepto cambia más, porque los outliers tienen valores y extremos que desplazan la recta verticalmente tanto como la inclinan.

3. Comparación de métricas MAE, MSE, RMSE y R^2

Métrica	Sin outliers	Con outliers	Factor de cambio
MAE	~19.1	~93.5	×4.9
MSE	~634	~45 022	×71
RMSE	~25.2	~212.2	×8.4
R^2	~0.903	~0.174	cae drásticamente

Todas las métricas empeoran, pero el **MSE** es, con diferencia, la que más se ve afectada.

4. ¿Por qué MSE/RMSE es más sensible a outliers que MAE?

El MSE eleva al cuadrado cada residuo, de modo que un error grande (p.ej. $10\times$) contribuye 100 veces más al MSE que un error unitario. El MAE suma los valores absolutos sin elevar al cuadrado, por lo que los errores grandes pesan proporcionalmente menos en comparación. Los outliers generan residuos muy grandes, que disparan el MSE de forma desproporcionada.

5. ¿Todos los outliers afectan igual al modelo?

No. Los valores de x extremos, lejos de $\bar{x}$ tienen mayor capacidad de rotar la recta y son más influyentes. Los outliers en y pero en zonas de x cercanas a la media solo desplazan verticalmente. En este caso, como los índices atípicos se escogen aleatoriamente de los 100 puntos, algunos caen en regiones de alto x (alto leverage) y afectan más la pendiente.

6. ¿El modelo aprende la tendencia general o es arrastrado por puntos extremos?

Con outliers, el modelo es claramente **arrastrado por los puntos extremos**: la pendiente crece de 3.025 a 3.978 y el R^2 cae de 0.90 a 0.17, lo que indica que el modelo ha perdido capacidad explicativa sobre el conjunto completo de datos. La recta ya no representa bien la tendencia de la mayoría de las observaciones.

7. En un dataset real, ¿eliminarías los outliers automáticamente?

No sería prudente eliminarlos de forma automática. Antes de decidir, habría que: - **Verificar si son errores** (entrada de datos incorrecta, fallo del instrumento de medición). - **Evaluar si son observaciones legítimas** que simplemente son casos raros pero reales.

Eliminar datos sin justificación sólida puede introducir sesgo y pérdida de información valiosa.

```
[17]: X_resaped = x.reshape(-1, 1)
```

```
[18]: model_outliers= linear_model.LinearRegression()  
model_outliers.fit(X_resaped, yp_wo)
```

```
[18]: LinearRegression()
```

Del resultado del modelo lineal para estos datos con outliers, guarde los coeficientes encontrados en un array llamado `theta_ne`

```
[19]: theta_ne = np.array([model_outliers.intercept_, model_outliers.coef_[0]])  
print(f"Theta ne = {theta_ne}")
```

```
Theta ne = [1.54811881 3.97842184]
```

Esto es la solución usando el modelo y la **ecuación normal**

1.1.6 Ahora implementaremos 3 métodos de gradient descent: batch, estocástico y mini-batch

En esta sección ajustamos una regresión lineal usando Gradient Descent en vez de usar directamente `LinearRegression`. Compararemos con los resultados encontrados en la regresión con outliers.

Agregaremos $x_0 = 1$ a cada instancia, este es el término de bias (β_0) y se usa para escribir la solución en la forma de multiplicación de matrices

$$y = X \cdot \theta$$

donde θ es el vector de coeficientes que incluye el término β_0 (término de bias) y $\beta_1, \beta_2, \beta_3, \dots, \beta_n$

```
[20]: X = np.c_[np.ones((100, 1)), x]  
  
print(X.shape) #la forma es el número de instancias x número de parámetros  
  
(100, 2)
```

Podemos calcular la pérdida asociada con la ecuación normal, usando la expresión

$$\text{MSE} = \frac{1}{m} \|X \cdot \theta - y\|^2$$

```
[21]: loss_ne = np.mean((X.dot(theta_ne) - yp_wo.reshape(-1,1))**2)
```

1.1.7 Batch GD

```
[22]: np.random.seed(10)  
  
eta = 0.0001 #learning step (valor pequeño asegura convergencia, aunque lenta)  
n_iterations = 1000 #puede cambiar este valor!!  
m = 100 #número de instancias
```

```

theta_path_bgd = [] #arreglo para guardar los valores de los parámetros en cada
↪ iteración

theta = np.random.randn(2,1) #inicializamos con valores aleatorios

for iteration in range(n_iterations):
    gradients = 2/m * X.T.dot(X.dot(theta) - yp_wo.reshape(-1,1)) #gradiente de
    ↪ la función de pérdida respecto a theta
    theta = theta - eta * gradients #actualiza los valores de theta, en cada
    ↪ cálculo del gradiente, en cada paso
    theta_path_bgd.append(theta) #

theta_path_bgd = np.array(theta_path_bgd) #guardamos esos valores (útil para
    ↪ una visualización)

theta_bgd = theta #resultado final

```

```
[23]: theta_bgd
```

```
[23]: array([[1.38909891],
           [3.98081931]])
```

```
[24]: loss_bgd = np.sum(1/m*(X.dot(theta_bgd) - yp_wo.reshape(-1,1))**2)
```

```
[25]: loss_bgd
```

```
[25]: np.float64(43259.08804185896)
```

```
[26]: (loss_ne-loss_bgd)/loss_ne*100 #diferencia porcentual con la ecuación normal
```

```
[26]: np.float64(37.87842821341625)
```

1.1.8 GD estocástico

```

[27]: np.random.seed(10) #

theta = np.random.randn(2,1) #inicializa los parámetros

eta = 0.000005 #learning step, valor más pequeño evita saltos muy grandes

n_iterations = 10000 #necesitamos más iteraciones en este caso

theta_path_sgd = []

for epoch in range(n_iterations):

    random_index = np.random.randint(m) #índice aleatorio

```

```

x_one = X[random_index:random_index+1] #sólo se selecciona una
↪ instancia del conjunto de datos (m es la cantidad total)

y_one = yp_wo[random_index:random_index+1]

gradients = 2 * x_one.T.dot(x_one.dot(theta) - y_one)
theta = theta - eta * gradients
theta_path_sgd.append(theta)

theta_path_sgd = np.array(theta_path_sgd)

theta_sgd = theta

```

```
[28]: theta_sgd
```

```
[28]: array([[1.3552955 ],
           [4.17721319]])
```

```
[29]: loss_sgd = np.sum(1/m*(X.dot(theta_sgd) - yp_wo.reshape(-1,1))**2)
```

```
[30]: loss_sgd
```

```
[30]: np.float64(43385.08132655123)
```

```
[31]: (loss_ne-loss_sgd)/loss_ne*100 #diferencia porcentual con la ecuación normal
```

```
[31]: np.float64(37.69749742559971)
```

1.1.9 Mini batch GD

```

[32]: # See also implementation notes here: https://sebastianraschka.com/faq/docs/
↪ sgd-methods.html

np.random.seed(10)

theta = np.random.randn(2,1)

eta = 0.000005

n_iterations = 1000

theta_path_mgd = []

minibatch_size = 10

for epoch in range(n_iterations):

```

```

    shuffled_indices = np.random.permutation(m) #se desordena el arreglo para
    ↪ seleccionar distintos mini-batches

    X_shuffled = X[shuffled_indices]

    y_shuffled = yp_wo.reshape(-1,1)[shuffled_indices]

    xi = X_shuffled[:minibatch_size] #subset aleatorio para calcular el
    ↪ gradiente

    yi = y_shuffled[:minibatch_size]

    gradients = 2/minibatch_size * xi.T.dot(xi.dot(theta) - yi)

    theta = theta - eta * gradients

    theta_path_mgd.append(theta)

theta_path_mgd = np.array(theta_path_mgd)

theta_mgd = theta

print(theta_mgd)

```

```

[[1.38191988]
 [4.25542902]]

```

```
[33]: theta_mgd
```

```
[33]: array([[1.38191988],
           [4.25542902]])
```

```
[34]: loss_mgd = np.sum(1/m*(X.dot(theta_mgd) - yp_wo.reshape(-1,1))**2)
```

```
[35]: loss_mgd
```

```
[35]: np.float64(43506.50418469318)
```

```
[36]: (loss_ne-loss_mgd)/loss_ne*100 #diferencia porcentual con la ecuación normal
```

```
[36]: np.float64(37.52312993104462)
```

Comparación de GD Veamos el camino que siguió cada método de GD que implementamos. El color más oscuro indica pasos posteriores

```
[37]: plt.figure(figsize=(14,8))
```

```

plt.scatter(theta_path_sgd[:,0].flatten(), theta_path_sgd[:,1].
    ↪flatten(), marker = 's', s = 5, \
        label="Stochastic GD, N$_{it}$ = 10000", c = np.arange(1000), cmap=plt.
    ↪cm.Purples)
plt.scatter(theta_path_mgd[:,0].flatten(), theta_path_mgd[:,1].flatten(),
    ↪marker = "+", s = 12, linewidth=1, \
        label="Mini-batch GD, N$_{it}$ = 1000", c = np.arange(1000),
    ↪cmap=plt.cm.Greens)
plt.scatter(theta_path_bgd[:,0].flatten(), theta_path_bgd[:,1].flatten(),
    ↪marker = "d", s = 12, linewidth=1, \
        label="Batch GD, N$_{it}$ = 1000", c = np.arange(1000,0,-1),
    ↪cmap=plt.cm.copper)

plt.scatter(theta_sgd[0],theta_sgd[1], marker = "s", s = 100, color = 'Purple',
    ↪alpha = 0.5)
plt.scatter(theta_mgd[0],theta_mgd[1], marker = "+", s = 200, color =
    ↪'DarkGreen', alpha = 1)
plt.scatter(theta_bgd[0],theta_bgd[1], marker = "d", s = 100, color = 'k',
    ↪alpha = 0.5)

legend = plt.legend(loc="upper left", fontsize=16)

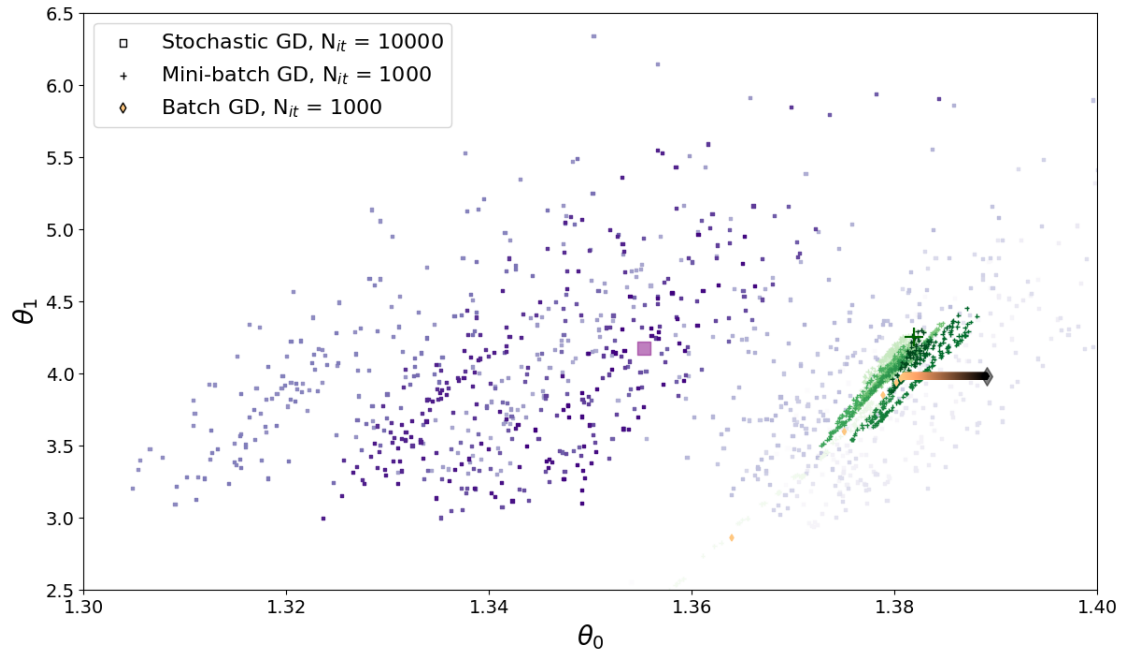
for i in range(3):
    legend.legend_handles[i].set_color('k')
    legend.legend_handles[i]._sizes = [30]

plt.xlabel(r"$\theta_0$", fontsize=20)
plt.ylabel(r"$\theta_1$", fontsize=20)

plt.axis([1.3, 1.4, 2.5, 6.5])

#plt.savefig('AllThePaths.png', dpi = 300)
plt.show()

```



1.1.10 Preguntas

- Identifica en el código de mini batch GD dónde ocurre cada paso:
 - cálculo de las predicciones
 - cálculo del error o residuo
 - cálculo del gradiente
 - actualización de los parámetros
- ¿Qué representan los parámetros θ_0 y θ_1 en este modelo?
- Compara los parámetros encontrados por Gradient Descent con los obtenidos mediante `LinearRegression`. ¿Son iguales o parecidos? ¿Por qué no necesariamente coinciden exactamente?
- Cambia el learning rate η . Prueba al menos tres valores: uno pequeño, uno razonable y uno demasiado grande. ¿Qué ocurre en cada caso?
- Explica con tus palabras por qué la actualización tiene un signo menos: $\theta \leftarrow \theta - \eta \nabla L$
- En la solución analítica derivamos e igualamos a cero. En Gradient Descent no resolvemos directamente la ecuación. ¿Qué hacemos en cambio?
- Compara Batch Gradient Descent, Stochastic Gradient Descent y Mini-batch Gradient Descent.
 - ¿Cuál usa todos los datos en cada actualización?
 - ¿Cuál actualiza los parámetros con más ruido?
 - ¿Cuál parece converger de forma más suave?

1. Identificación de pasos en el código Mini-batch GD

```
[39]: # Cálculo de las predicciones:
xi.dot(theta)          #  $\hat{y} = X_{mini}$ 

# Cálculo del error o residuo:
xi.dot(theta) - yi      # residuo =  $\hat{y} - y$ 

# Cálculo del gradiente:
gradients = 2/minibatch_size * xi.T.dot(xi.dot(theta) - yi)  #  $L()$ 

# Actualización de los parámetros:
theta = theta - eta * gradients  #  $\theta = \theta - \eta \cdot L$ 

print(f"Predicción: {xi.dot(theta)}")
print(f"Valor real: {yi}")
print(f"Gradiente: {gradients}")
print(f"Actualización de los parámetros: {theta}")
```

```
Predicción: [[ 43.83565045]
 [345.25481094]
 [ 56.57167131]
 [ 65.06235189]
 [ 69.30769218]
 [ 94.77973391]
 [162.70517853]
 [111.76109507]
 [ 35.34496987]
 [260.34800517]]
Valor real: [[ 9]
 [252]
 [183]
 [ 42]
 [ 37]
 [445]
 [ 95]
 [ 93]
 [ 35]
 [220]]
Gradiente: [[ -32.91554064]
 [1001.91950103]]
Actualización de los parámetros: [[1.38224756]
 [4.24534029]]
```

1.1.11 Respuestas

2. ¿Qué representan θ_0 y θ_1 ?

- θ_0 es el **intercepto** (β_0): el valor esperado de y cuando $x = 0$.

- θ_1 es la **pendiente** (β_1): el cambio promedio de y por unidad de x .

Son exactamente los mismos parámetros que `model.intercept_` y `model.coef_[0]` de `LinearRegression`.

3. Comparación con `LinearRegression`

Método	θ_0	θ_1
Ecuación normal	1.548	3.978
Batch GD	1.389	3.981
SGD	1.355	4.177
Mini-batch GD	1.382	4.255

Los valores son **similares pero no exactamente iguales** al resultado analítico. GD es un método iterativo que requiere suficientes iteraciones y un learning rate adecuado para converger. Con más iteraciones o un η ajustado, la diferencia disminuiría. Además, SGD y mini-batch son inherentemente ruidosos (usan subconjuntos de datos), por lo que oscilan alrededor del mínimo en lugar de converger exactamente.

4. Efecto del learning rate η

- η **muy pequeño** (e.g. 10^{-6}): el algoritmo da pasos ínfimos y necesita muchísimas iteraciones para converger; con las iteraciones fijas del código, los parámetros apenas se mueven desde la inicialización.
- η **razonable** (e.g. 10^{-4} para Batch GD): convergencia estable y progresiva hacia el mínimo; la pérdida disminuye en cada iteración.
- η **demasiado grande** (e.g. 0.01): el gradiente “sobreajusta” cada paso, los parámetros oscilan de un lado al otro y la pérdida puede crecer sin límite (divergencia).

5. ¿Por qué la actualización tiene signo menos: $\theta \leftarrow \theta - \eta \nabla L$?

El gradiente ∇L apunta en la **dirección de mayor crecimiento** de la función de pérdida. Para *minimizar* la pérdida queremos movernos en la dirección opuesta, es decir, **cuesta abajo**. El signo negativo garantiza que cada actualización reduce (o al menos no aumenta) la pérdida, dirigiendo los parámetros hacia el mínimo.

6. Diferencia entre solución analítica y Gradient Descent

En la solución analítica (ecuación normal) se deriva la función de pérdida respecto a θ , se iguala a cero y se resuelve el sistema directamente: $\theta = (X^T X)^{-1} X^T y$. Es una solución **exacta en un solo paso**, pero requiere invertir una matriz (costoso para muchas features).

En Gradient Descent **no se resuelve la ecuación directamente**: se parte de unos valores iniciales aleatorios y se actualizan iterativamente siguiendo la dirección del gradiente negativo, acercándose

al mínimo de forma incremental. Es aproximado pero escala bien a grandes conjuntos de datos y modelos con muchos parámetros.

7. Comparación Batch GD vs. SGD vs. Mini-batch GD

Característica	Batch GD	SGD	Mini-batch GD
Datos usados por actualización	Todos (100)	Uno	Subconjunto (10)
Ruido en la actualización	Bajo (suave)	Alto (ruidoso)	Intermedio
Velocidad por iteración	Lenta	Rápida	Intermedia
Convergencia	Suave y estable	Ruidosa, oscila	Más suave que SGD
Riesgo de quedar en mínimos locales	Mayor	Menor (el ruido ayuda a escapar)	Intermedio

En el gráfico del espacio de parámetros se observa que: - **Batch GD** traza un camino limpio y directo hacia el mínimo. - **SGD** muestra un camino muy errático con muchos saltos. - **Mini-batch GD** es intermedio: avanza con cierto ruido pero más suavemente que SGD.